Implementation Document

for

Real-Time Multi-Vendor Food Ordering and Delivery Management System for a University Campus

Version 1.0

Prepared by

Group 24: **Group Name:** AlgoMinds

Aarsh Kaushik	220014	Aarshk22@iitk.ac.in
Vikas Sani	231151	Svikas23@iitk.ac.in
Kajal	230513	Kajalb23@iitk.ac.in
Banoth Ramcharan	240259	Ramcharanb24@iitk.ac.in
Ritik Ahirwar	210862	ritika21@iitk.ac.in
Banoth Venkatamaniram	240260	banothv24@iitk.ac.in
Abhimanyu Singh Rawat	240029	abhimanyur24@iitk.ac.in
Ashish Kumar Soni	230228	sashish@iitk.ac.in
Pravin Tanpure	220815	pravint22@iitk.ac.in

Course: CS253

Mentor TA: Aditya Ojha

Date: 27/03/2026

CONTENTS II
REVISIONS II

1 IMPLEMENTATION DETAILS 1

2 CODEBASE 2

3 COMPLETENESS 3

APPENDIX A - GROUP LOG 4

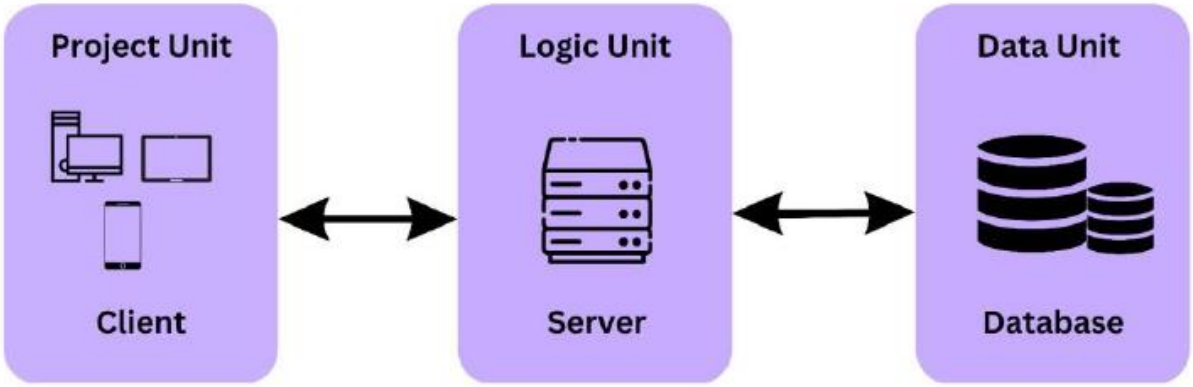
Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Aarsh Kaushik Vikas Saini Kajal Banoth Ramcharan Ritik Ahirwar Banoth Venkatamaniram Abhimanyu Singh Rawat Ashish Kumar Soni Pavin Tanpure	First Draft.	27/03/26

1 Implementation Details

AlogMinds implements a three-Unit architecture, a popular pattern for user-interactive applications.
The Unit that comprises this architecture includes:

- Project Unit: This represents the components users directly interact with. In our case, this is an HTML, CSS, JavaScript-based web application that runs inside a browser.
- Logic Unit: This contains the code required to translate user actions to the functionality, written in Python, that drives the application's behaviour at the project unit. Djanjo framework has been used on top of Python and JavaScript.
- Data Unit: This consists of a database that holds the data relevant to the application.



1.1 Project Unit:

In the realm of web development, the user interface layer serves as the gateway for users to engage with a system via their web browsers. This critical layer incorporates various components designed to facilitate seamless interactions and intuitive navigation, ensuring that users can effortlessly access and utilize the system's features. It plays a pivotal role in presenting information in a clear and understandable manner, thereby enhancing the overall user experience.

Our web development endeavour revolves around harnessing the power of three foundational technologies: HTML, CSS, and JavaScript. HTML lays the groundwork by structuring the content of web pages, providing a backbone upon which information is organized and presented. CSS steps in to define the visual aesthetics and stylistic elements of the website, shaping its appearance and enhancing its appeal. Meanwhile, JavaScript, bolstered by its accompanying frameworks, injects dynamism and interactivity into the web environment, enabling real-time responses to user inputs and events, thus elevating the web experience to new heights of engagement and functionality.

1.2 Logic/Reasoning Unit:

The Reasoning Level handles user queries from the web browser, conducts computations on them, and determines the routes for data retrieval. It's responsible for scripting the processes through which requests and data flow in the backend.
The logic tier is implemented in **Python**, one of the most widely used and easiest to use programming languages

- **Easy to Learn:** Python's simplicity and intuitiveness make it a favoured choice for both beginners and seasoned developers.
- **Large Standard Library**: Python comes with a vast standard library, providing pre-built functions and modules for a wide range of tasks, saving developers time and effort.
- **Versatile**: Python can be used for various applications, from web development to scientific computing, data analysis, machine learning, and more.
- **Third-party Libraries**: Python has a large and active community that creates third-party libraries for almost any task, making it easy to extend its capabilities and reduce development time.
- **Interpreted**: Python is an interpreted language, which means it doesn't require compilation before running, making it faster to write and test code, ideal for prototyping and iterative development.

The framework used on top of Python is **Django**. Django is a high-level Python web framework for building web applications quickly and efficiently.

The advantages of using Django are:

- **Faster Development**: Django's pre-built components and conventions can help developers build web applications quickly.

- **Scalability**: Django is scalable and can handle large and complex web applications with ease.
- **Robust Security**: Django has built-in security features that protect against common web application vulnerabilities.
- **Versatility**: Django can be used for a variety of web applications, from content-based sites to complex web applications.
- **Active Community**: Django has a large and active community that provides support, resources, and third-party packages to extend its capabilities.
- **Built-in Administration Interface**: Django's built-in admin interface provides a powerful and customizable tool for managing site content, user accounts, and other site features, without requiring developers to build custom admin interfaces.

1.3 Data Management Unit:

The Data Tier, also known as the Database Tier, is responsible for storing, managing, and retrieving all the data processed by the application. It ensures data consistency, integrity, and efficient access for various operations performed by the system.

In this project, PostgreSQL is used as the primary database management system. PostgreSQL is a powerful, open-source relational database known for its reliability, scalability, and advanced features. It efficiently handles structured data and supports complex queries, making it suitable for modern web applications.

The main advantages and benefits of PostgreSQL are:

● Open Source: PostgreSQL is open-source and freely available, allowing developers to use, modify, and distribute it without licensing restrictions.

● High Reliability and ACID Compliance: PostgreSQL ensures data integrity through full support for ACID (Atomicity, Consistency, Isolation, Durability) transactions, making it highly reliable for critical applications.

● Scalability: PostgreSQL can handle large volumes of data and supports scaling for high-performance applications, making it suitable for both small and enterprise-level systems.

● Advanced Features: It supports complex queries, indexing, triggers, stored procedures, and JSON data types, providing flexibility for diverse use cases.

● Strong Security: PostgreSQL includes robust authentication, access control, and encryption features to protect sensitive data.

● Cross-Platform Support: PostgreSQL works seamlessly across multiple operating systems such as Windows, Linux, and macOS.

● Extensibility: It allows developers to extend its functionality through custom functions, extensions, and plugins.

● Integration with ORMs: PostgreSQL integrates well with ORMs like Sequelize, simplifying database operations and reducing development complexity.

Overall, PostgreSQL provides a robust, scalable, and secure data management solution, ensuring efficient handling of application data and supporting the overall performance of the system.

1.4 Development and version control environment:

We utilized **Git** as our version control system, which is widely recognized as the most prevalent system employed for software development and other version control activities. Git is designed to keep track of modifications made to files, providing a comprehensive record of all changes made. This functionality permits users to revert to particular versions if needed. Moreover, Git simplifies collaboration, enabling numerous individuals to combine their modifications into a single source. To facilitate repository management and collaboration among ourselves, we employed **GitHub**, a web-based Git repository hosting service.

1.5 External API Used :

a) Google Maps APIs (Places & Geocoding)
Used for handling location-based features such as vendor locations, student addresses, and delivery routing.
Justification:

- Highly accurate and reliable location data
- Easy-to-use APIs with good documentation
- Real-time geocoding capabilities

Compared to alternatives like OpenStreetMap, Google Maps provides better accuracy, richer data, and more robust support.

b) Razorpay Payment Gateway
Used for handling online payments within the system.
Justification:

- Popular and widely trusted in India
- Easy integration with Python/Django
- Supports multiple payment methods (UPI, cards, net banking)
- Strong security and compliance

Compared to alternatives like Paytm or Stripe, Razorpay offers better local support and seamless integration for Indian users.

c) Gmail SMTP (Email Service)
Used for sending emails such as registration confirmation, login details, and order updates.
Justification:

- Simple and reliable email delivery
- No need for complex email server setup
- Easily integrated using SMTP libraries in Python

Compared to services like SendGrid or Mailgun, Gmail SMTP is easier to configure and sufficient for small to medium-scale applications.

The application was developed and tested using Django's built-in development server. This approach allows quick testing and debugging without requiring additional deployment configuration.

2 Codebase

An organization has been set up on GitHub for storing and collaborating on the source code of this project, AlgoMinds.

```
project_root/
│
├── backend/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
│
├── media/
│   ├── documents
│   └── menu_photos
│
├── templates/
│   ├── delivery
│   ├── student
│   ├── users
│   └── vendor
│
├── users  (Django app)
├── .gitignore
├── QUICK_START.md
├── README.md
├── STUDENT_DASHBOARD_SETUP.md
├── manage.py
└── requirements.txt
```

3 Completeness

The implemented system fulfils the major functional and non-functional requirements outlined in the Software Requirements Specification. The system provides a complete workflow for users, vendors, and administrators to interact efficiently within a campus-based food ordering and delivery environment.

Functional Requirements Achieved

The following features have been successfully implemented:

User Interface (Customer Side)

- User Registration and Login
- Browsing available vendors and menus
- Searching for food items and vendors
- Adding items to cart
- Placing orders
- Viewing order history
- Real – time order status tracking

Vendor Interface

- Vendor Registration and Login
- Menu Management (Add/Edit/Delete items)
- Viewing incoming orders
- Updating order status (Accepted, Preparing, Completed)
- Managing availability of items

Order Management System

- User Registration and Login
- Browsing available vendors and menus
- Searching for food items and vendors
- Adding items to cart
- Placing orders
- Viewing order history
- Real-time order status tracking

System Features

- Efficient handling of multiple users and vendors
- Dynamic updates to menus and availability

- Structured data storage and retrieval using a database

Non-Functional Requirements Achieved

Security Requirements

- Authentication**:**
  All users (customers and vendors) must log in using unique credentials.
- Authorisation**:**
  Access control ensures that users can only perform actions permitted for their role (customer/vendor).
- Data Protection**:**
  Sensitive information, such as passwords, is stored securely using hashing techniques.

Safety and Reliability

- The system ensures that only authenticated users can place or manage orders.
- Data consistency is maintained during order placement and updates.
- The system prevents unauthorised access to user or vendor data.

Performance

- The system is capable of handling multiple concurrent users.
- Efficient database queries ensure quick response time for menu loading and order processing

Usability

- The interface is designed to be simple and intuitive.
- Users can easily navigate between vendors, menus, and order tracking pages.

Software Quality Attributes

The system satisfies key quality attributes:

- Reliability**:** Stable performance under normal usage conditions
- Scalability**:** Can be extended to support more users and vendors
- Maintainability**:** Modular design allows easy updates and feature additions
- Usability**:** Clean and user-friendly interface

Current Limitations and Future Improvements

While the system meets the core requirements, the following enhancements are planned:

- Real-time notifications for order updates
- Advanced search and filtering options
- Mobile application for improved accessibility

- Rating and review system for vendors

Appendix A - Group Log

- We have a WhatsApp group for effective communication.
- We have created a private organization on GitHub so that we can work on the product with collaboration.
- We have made a plan of who in the team will handle different parts of the project like design, frontend, backend, and database.

S. No.	DATE	TIMINGS	VENUE	DESCRIPTION
1.	10-02-2026	20:00-21:00	RM 3rd floor	Discussed and decided who will work on the backend and Frontend.
2.	18-02-2026	20:00-21:00	Google Meet	Distributed tasks to each member.
3.	01-03-2026	18:00-20:00	Google Meet	Reviewed the progress made till now and discussed the changes that needed to be implemented.
4.	12-03-2026	21:00-22:00	CCD	All team members meet to review the initial draft of SID and implement the necessary changes based on the feedback provided.
5.	24-03-2026	20:00-21:00	RM 3rd floor	All-team meet to review the final changes in the document